

Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 22.Б09-мм

Поддержка многоядерности в ОСРВ Embox

Николаев Даниил Владимирович

Отчёт по учебной практике
в форме «Производственное задание»

Научный руководитель:
доцент кафедры системного программирования, к.ф.-м.н., Луцив Д.В.

Консультант:
Бондарев А.В.

Санкт-Петербург
2024

Оглавление

Введение	3
1. Постановка задачи	4
2. Обзор	5
2.1. АМР и SMP архитектуры	5
2.2. Embox	5
2.3. FreeRTOS	6
2.4. QNX	7
3. Реализация	8
3.1. Разработка собственной стратегии планирования	8
3.2. Многоядерность в Embox	8
4. Тестирование	10
Заключение	12
Список литературы	13

Введение

Современная архитектура компьютерных систем прошла путь эволюции от одноядерных к многоядерным конфигурациям. Одноядерные архитектуры, хотя и просты в реализации, ограничены в своей производительности и все реже используются, за исключением специализированных приложений. Напротив, многоядерные архитектуры стали основой современных вычислительных систем, обеспечивая значительное улучшение производительности за счет параллельной обработки данных[4].

В контексте ОСРВ, таких как Embox[1], переход к многоядерности представляет собой значительный шаг вперед, однако он также влечет за собой сложности в оптимизации и управлении. Эффективная работа в многоядерной среде требует разработки новых стратегий планирования задач, синхронизации и управления ресурсами. Для Embox это особенно актуально, учитывая текущие ограничения в поддержке многоядерных архитектур.

Целью данной работы является исследование и разработка методов для улучшения поддержки многоядерности в Embox. Основные вопросы касаются того, какие стратегии планирования и синхронизации наиболее эффективны в многоядерной среде и как их можно интегрировать в Embox для повышения производительности и надежности.

План работы включает анализ существующих подходов к многоядерной обработке в Embox, а также изучение передовых стратегий, применяемых в других ОСРВ. На основе этого анализа будут разработаны усовершенствованные методы планирования и синхронизации, адаптированные к многоядерной архитектуре Embox.

1. Постановка задачи

Целью работы является улучшение планировщика Embox для работы в многоядерном режиме

Задачи:

- Исследование многоядерности и многозадачности в Embox.
- Обзор подходов к обеспечению многоядерности.
- Разработка собственной стратегии планирования с поддержкой многоядерности в ОСРВ Embox.
- Апробация.

2. Обзор

2.1. АМР и SMP архитектуры

В современных многоядерных архитектурах преобладают два основных типа: Симметричная многопроцессорность (SMP) и Асимметричная многопроцессорность (АМР). Они различаются по способу организации и взаимодействия процессоров с памятью и операционной системой.[?] В системах SMP все процессоры имеют общий доступ к одной и той же памяти и работают с одной копией операционной системы. Это означает, что каждый процессор исполняет один и тот же код ОС, что обеспечивает удобство управления и синхронизации, но также требует тщательной координации доступа к общей памяти.

С другой стороны, системы АМР характеризуются тем, что каждый процессор работает с собственным экземпляром операционной системы и обладает индивидуальной областью памяти. Это уменьшает потребность в синхронизации, поскольку каждый процессор работает независимо. Однако, это также вносит сложности в проектирование системы, поскольку требует специальных механизмов для обмена данными между процессами, выполняющимися на разных процессорах.

Кроме того, асимметричная организация АМР создает определенные трудности в поддержке и обновлении операционной системы, поскольку изменения необходимо реализовывать для каждого процессора индивидуально, учитывая особенности каждой изолированной системы.[?]

2.2. Embox

В Embox[1] реализованы три стратегии планирования: `trivial`, `priority_based_smp` и `prioq`. Каждая из них ориентирована на определенные условия работы системы и типы задач:

`Trivial` — простая стратегия, реализованная через `list.c` и `list.h`, предполагает базовое планирование без учета приоритетов. `Prioq` — стандартная стратегия, использующая приоритеты задач, реализована через `prioq.c` и `prioq.h`. `Priority_based_smp` — стратегия, оптимизиро-

ванная для работы в многоядерных системах, использует `list_array.c` и `list_array.h`.

Последняя стратегия имеет архитектуру симметричной многопроцессорности. Как происходит выбор задачи? Когда система выбирает задачу для выполнения, она начинает поиск с задач самого высокого приоритета. Система проверяет аффинитет каждой задачи, то есть определяет, подходит ли текущий CPU для выполнения этой задачи.

Если задача имеет аффинитет к текущему CPU (или группе CPU), то она может быть выбрана для выполнения на этом CPU. Если аффинитет не совпадает, система продолжает поиск среди других задач в очереди.

Однако, в контексте многоядерности, у Embox имеются определенные недостатки. Одним из основных является то, что поддержка многоядерности ограничена архитектурой x86. Кроме того, в Embox используется единая очередь для всех ядер, что может привести к снижению эффективности распределения задач между ядрами, особенно в условиях высокой загрузки или сложных многопоточных операций.

2.3. FreeRTOS

FreeRTOS[2] является одной из наиболее популярных операционных систем реального времени, активно используемой во множестве встроенных систем. Основное преимущество FreeRTOS заключается в её универсальности и масштабируемости, что позволяет ей эффективно функционировать как на одноядерных, так и на многоядерных архитектурах.

AMP:

- каждое ядро многоядерного устройства запускает свой собственный независимый экземпляр FreeRTOS
- используется буфер сообщений в качестве примитива межъядерной связи, чтобы задачи на одном ядре могли переходить в заблокированное состояние для ожидания данных или событий, отправленных с другого ядра.

SMP:

- один экземпляр FreeRTOS планирует задачи системы на нескольких процессорных ядрах. Каждое ядро должно иметь одинаковую архитектуру процессора и совместно использовать одно и то же пространство памяти.

Эффективность FreeRTOS в обоих режимах делает её важным объектом для изучения в контексте многозадачности и многоядерности.

2.4. QNX

QNX[3], высоконадежная операционная система реального времени, предназначена в первую очередь для использования в встраиваемых системах и выделяется своей стабильностью и производительностью в многоядерных конфигурациях.[?]

SMP:

- Запускаются потоки с наивысшим приоритетом на всех доступных ядрах процессора.
- Для снижения частоты миграции потоков между ядрами, что может приводить к увеличению времени доступа к кэшу и снижению производительности, SMP планировщик QNX стремится выполнять поток на том же ядре, на котором он выполнялся в последний раз. Это повышает эффективность использования кэша процессора и снижает накладные расходы на переключение контекста.
- Некоторые процессы могут быть привязаны к конкретным ядрам (CPU Affinity)

3. Реализация

3.1. Разработка собственной стратегии планирования

Собственная стратегия планирования была разработана с учетом приоритетов задач. Код стратегии представлен в файлах `custom_scheduler.c` и `custom_scheduler.h`. Основное внимание уделено эффективному распределению ресурсов между задачами с различными приоритетами.

Для запуска этой стратегии необходимо в файле конфигураций `mods.conf` изменить стратегию планирования с `priority_based` на `custom_sched`

На данный момент стратегия оптимизирована лишь для одноядерной архитектуры

3.2. Многоядерность в Embox

В рамках исследования была рассмотрена многоядерность в Embox. Особое внимание уделено стратегии `priority_based_smp`, оптимизированной для многоядерных систем. Проведен эксперимент с подключением двух ядер, демонстрирующий параллельную работу задач на разных ядрах.

Листинг 1: Пример использования многоядерности в Embox

```
static volatile int shared_counter = 0;
static void *thread_func(void *arg) {
    int iterations = 10;
    for (int i = 0; i < iterations; i++) {
        int local_counter = ++shared_counter;
        printf("CPU %d, iteration %d, shared_counter: %d\n" \
            ,cpu_get_id(), i, local_counter);
    }
    return NULL;
}
```



```

void start_thread_on_cpu(int cpu_id) {
    struct thread *t = thread_create(THREAD_FLAG_SUSPENDED, \
    thread_func, NULL); // создаем поток
    cpu_bind(cpu_id, t); //привязываем поток к ядру
    thread_launch(t); //запускаем поток
}

```

Для того, чтобы использовать Embox в многоядерном режиме, необходимо собрать Embox с помощью шаблона x86/smp и после компиляции запустить на эмуляторе QEMU: `./scripts/qemu/auto_qemu -smp 2` (где 2 — это количество ядер)

Разработка собственной стратегии планирования задач, оптимизированной для одноядерной архитектуры, а также проведение исследований многоядерности в Embox, являются ключевыми этапами на пути к достижению основной цели проекта. Эта цель заключается в значительном улучшении многоядерного планирования в Embox, что позволит системе более эффективно использовать многоядерные архитектуры.

4. Тестирование

```
[info] test: running embox.test.stdio.printf_test
[info] test: running embox.test.recursion
[info] test: running embox.test.critical
[info] unit: initializing embox.fs.rootfs_oldfs
[info] runlevel: init level is 2
[info] runlevel: init level is 3
ACPI: RSDP 0x0000Default IO device[ttys0]
>export PWD=/
>export HOME=/
>partition
>service net_service
Starting service: net_service
Succesfully started
>service telnetd
Starting service: telnetd
Succesfully started
>service httpd
Starting service: httpd
Succesfully started
>source time_setup.sh

2021-08-31 19:02:40
>tish
root@embox: /#
```

Рис. 1: Сборка образа Embox

Как видно из Рисунка 1 собственная стратегия успешно проходит модульные тесты Embox

Эксперименты показали, что собственная стратегия планирования позволяет обрабатывать более 1900 задач (TASK_COUNT), в то время как стандартная стратегия `prioq` справляется только с 1600 задачами. Тестирование было проведено на платформе x86 в образе Embox. Embox был запущен на эмуляторе QEMU.

Листинг 2: Ручное тестирование очереди `runq_t` собственной стратегии планирования

```
#define TASK_COUNT 1900
```

```
void test_strategy_performance(void) {
    runq_t queue; // Очередь для хранения задач
    struct schedee tasks[TASK_COUNT]; // Массив задач
    clock_t start_time, end_time; // Переменные для замера времени

    runq_init(&queue); // Инициализация очереди
    for (int i = 0; i < TASK_COUNT; ++i) {
        // Инициализация каждой задачи
        runq_item_init(&tasks[i].runq_link);
    }
}
```

```
start_time = clock();
for (int i = 0; i < TASK_COUNT; ++i) {
    runq_insert(&queue, &tasks[i]);
}
end_time = clock();
printf("Insert time: %f seconds\n", \
(double)(end_time - start_time)/CLOCKS_PER_SEC);
// ...

}
```

Заключение

Результат

- Исследована многоядерность и многозадачность в Embox
 - Разработана и подключена собственная стратегия планирования
 - Проведены эксперименты на эмуляторе QEMU:
 - * сравнение стратегий Embox
 - * использование двух ядер
- Проведен обзор поддержки многоядерности в различных ОСРВ

[Ссылка на pull-request](#)

Список литературы

- [1] Embox source code repository. — URL: <https://github.com/embox/embox> (дата обращения: 1 мая 2024 г.).
- [2] FreeRTOS documentation website — URL: <https://www.freertos.org/index.html> (дата обращения: 1 мая 2024 г.).
- [3] QNX documentation website — URL: <https://www.qnx.com/developers/docs/am11/index.jsp> (дата обращения: 1 мая 2024 г.).
- [4] Design and Implementation of Multi-core Support for an Embedded Real-time Operating System for Space Applications — URL: <https://core.ac.uk/download/pdf/31017654.pdf> (дата обращения: 1 мая 2024 г.).
- [5] Asymmetric Multiprocessing — URL: <https://networkencyclopedia.com/asymmetric-multiprocessing-amp/> (дата обращения: 1 мая 2024 г.).
- [6] Running AMP, SMP or BMP Mode for Multicore Embedded Systems — URL: <https://caxapa.ru/thumbs/565353/PWRARBYNDBITSRAS.pdf> (дата обращения: 1 мая 2024 г.).
- [7] Таненбаум Э., Бос Х.. Современные операционные системы. — СПб.: Питер, 2015.